Experiment No. 10

Experiment Name - Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Select a standard dataset such as MNIST, Fashion-MNIST, EMNIST, or CelebA for the classification task.

Selected Dataset: MNIST (Modified National Institute of Standards and Technology) Handwritten Digit Dataset.

Description: The MNIST dataset contains 70,000 grayscale images of handwritten digits from 0 to 9. Each image has a resolution of 28 × 28 pixels. It consists of 60,000 training images and 10,000 testing images. This dataset is selected to develop and evaluate a CNN-based digit classification model.

2. Import the required libraries and load the selected dataset into the development environment.

```
# Import Required Libraries and Load Dataset
import numpy as np
import matplotlib.pyplot  as plt
import tensorflow  as tf
from tensorflow.keras.datasets  import mnist

# Load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data ()

# Display dataset information
print("Dataset loaded successfully!" )
print("Training images shape:" , X_train.shape )
print("Training labels shape:" , y_train.shape )
print("Testing images shape:" , X_test.shape )
print("Testing labels shape:" , y_test.shape )
print("Number of classes:" , len(np.unique(y_train)))
print("Classes:", np.unique(y_train))
```

```
Dataset loaded successfully!
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)
Number of classes: 10
Classes: [0 1 2 3 4 5 6 7 8 9]
```

3. Perform image preprocessing by resizing (if required), normalization, and splitting the dataset into training, validation, andtesting sets.

```
from sklearn.model_selection  import train_test_split

# Check image dimensions
print("Original image shape:" , X_train.shape [1:])

# Resize images if required
# MNIST images are already 28x28, so resizing is n ot required.

# Normalize pixel values from 0-255 to 0-1
X_train = X_train.astype ("float32") / 255.0
X_test = X_test.astype ("float32") / 255.0

# Split training data into training and validation  sets
X_train, X_val, y_train, y_val = train_test_split (
    X_train,
    y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

# Display dataset sizes
print("\nDataset split completed successfully!" )
print("Training images:" , X_train.shape )
print("Training labels:" , y_train.shape )
print("Validation images:" , X_val.shape )
print("Validation labels:" , y_val.shape )
print("Testing images:" , X_test.shape )
print("Testing labels:" , y_test.shape )
```

```
print("\nPixel value range after normalization:" ,
      X_train.min(), "to", X_train.max())
```

```
Original image shape: (28, 28)

Dataset split completed successfully!
Training images: (48000, 28, 28)
Training labels: (48000,)
Validation images: (12000, 28, 28)
Validation labels: (12000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)

Pixel value range after normalization: 0.0 to 1.0
```

4. Design a Convolutional Neural Network (CNN) consisting of convolutional layers, pooling layers, activation functions, fullyconnected layers, and an output layer.

```
# Design the Convolutional Neural Network (CNN)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Add channel dimension to the images
X_train = X_train[..., np.newaxis]
X_val = X_val[..., np.newaxis]
X_test = X_test[..., np.newaxis]

# Build the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),

    Dense(10, activation='softmax')
])

# Display the model architecture
model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

5. Configure the model using an appropriate optimizer, loss function, learning rate, batch size, and number of training epochs.

```
# Configure the CNN Model
from tensorflow.keras.optimizers  import Adam

# Configure the model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy' ,
    metrics=['accuracy']
)

# Set training parameters
batch_size = 64
epochs = 10

print("Model compiled successfully!" )
print("Optimizer: Adam" )
print("Learning rate: 0.001" )
print("Loss function: Sparse Categorical Crossentropy" )
print("Batch size:", batch_size)
print("Training epochs:", epochs)
```

```
Model compiled successfully!
Optimizer: Adam
Learning rate: 0.001
Loss function: Sparse Categorical Crossentropy
Batch size: 64
Training epochs: 10
```

6. Train the CNN model using the training dataset and monitor the learning process through training and validationaccuracy/loss curves.

```
# Train the CNN Model
import matplotlib.pyplot  as plt

# Train the model
history = model.fit(
    X_train,
    y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_data=(X_val, y_val),
    verbose=1
)

# Plot training and validation accuracy
plt.figure(figsize=(12, 5))

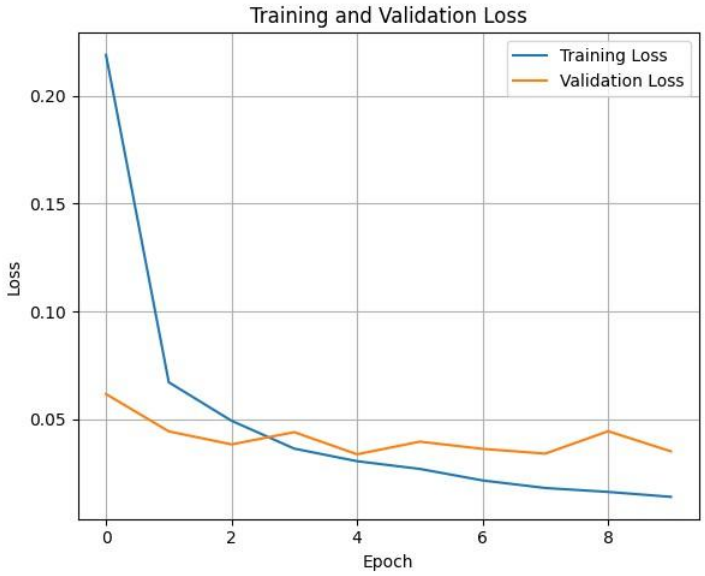
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Plot training and validation loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.9335 - loss: 0.2190 - val_accuracy: 0.9813 - val_loss: 0.0618
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 38s 51ms/step - accuracy: 0.9792 - loss: 0.0672 - val_accuracy: 0.9862 - val_loss: 0.0444
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 40s 53ms/step - accuracy: 0.9854 - loss: 0.0494 - val_accuracy: 0.9877 - val_loss: 0.0384
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 45s 58ms/step - accuracy: 0.9884 - loss: 0.0364 - val_accuracy: 0.9869 - val_loss: 0.0441
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 79s 55ms/step - accuracy: 0.9908 - loss: 0.0306 - val_accuracy: 0.9897 - val_loss: 0.0338
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.9913 - loss: 0.0270 - val_accuracy: 0.9891 - val_loss: 0.0397
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 83s 56ms/step - accuracy: 0.9928 - loss: 0.0217 - val_accuracy: 0.9893 - val_loss: 0.0363
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.9940 - loss: 0.0182 - val_accuracy: 0.9903 - val_loss: 0.0341
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 40s 53ms/step - accuracy: 0.9947 - loss: 0.0164 - val_accuracy: 0.9888 - val_loss: 0.0445
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 39s 52ms/step - accuracy: 0.9955 - loss: 0.0141 - val_accuracy: 0.9903 - val_loss: 0.0352
```





7. Evaluate the trained model on the testing dataset using performance metrics such as Accuracy, Precision, Recall, F1-Score,and Confusion Matrix.

```
# Evaluating the CNN Model
import numpy as np
import matplotlib.pyplot  as plt
import seaborn as sns
from sklearn.metrics  import (
    accuracy_score ,
    precision_score ,
    recall_score ,
    f1_score ,
    confusion_matrix ,
    classification_report
)

# Evaluate the model on the testing dataset
test_loss, test_accuracy = model.evaluate (X_test, y_test, verbose=0)

# Predict digit classes
y_pred_prob = model.predict (X_test, verbose=0)
y_pred = np.argmax (y_pred_prob , axis=1)

# Calculate performance metrics
accuracy = accuracy_score (y_test, y_pred)
precision = precision_score (y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score (y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

# Display performance metrics
print("CNN Model Evaluation Results" )
print("--------------------------------" )
print(f"Test Loss:  {test_loss:.4f}")
print(f"Accuracy:   {accuracy:.4f} ({accuracy * 100:.2f}%)")
print(f"Precision:  {precision:.4f}")
print(f"Recall:     {recall:.4f}")
print(f"F1-Score:   {f1:.4f}")

# Display classification report
print("\nClassification Report:" )
print(classification_report (y_test, y_pred, zero_division=0))

# Generate confusion matrix
cm = confusion_matrix (y_test, y_pred)
```

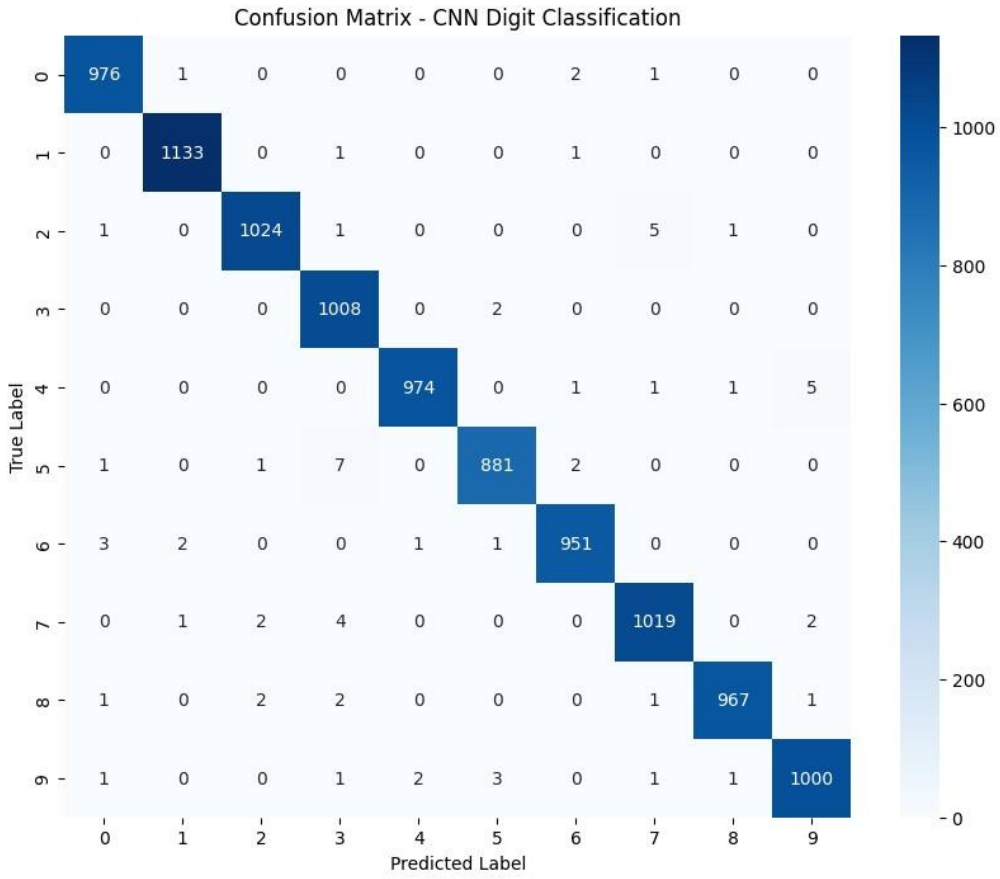
```
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - CNN Digit Classification' )
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
CNN Model Evaluation Results
--------------------------------
Test Loss:  0.0250
Accuracy:   0.9933 (99.33%)
Precision:  0.9933
Recall:     0.9933
F1-Score:   0.9933

Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       980
           1       1.00      1.00      1.00      1135
           2       1.00      0.99      0.99      1032
           3       0.98      1.00      0.99      1010
           4       1.00      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       1.00      0.99      0.99       974
           9       0.99      0.99      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```



8. Perform predictions on unseen images and compare the predicted labels with the actual labels.

```
# Predict Unseen Test Images and Compare Labels
import numpy as np
import matplotlib.pyplot as plt

# Select 10 unseen test images
num_images = 10
sample_images = X_test[:num_images]
actual_labels = y_test[:num_images]

# Predict the digit labels
predictions = model.predict(sample_images, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

# Display images with actual and predicted labels
plt.figure(figsize=(10, 6))

for i in range(num_images):
    plt.subplot(2, 5, i + 1)
    plt.imshow(sample_images[i].squeeze(), cmap='gray')
```

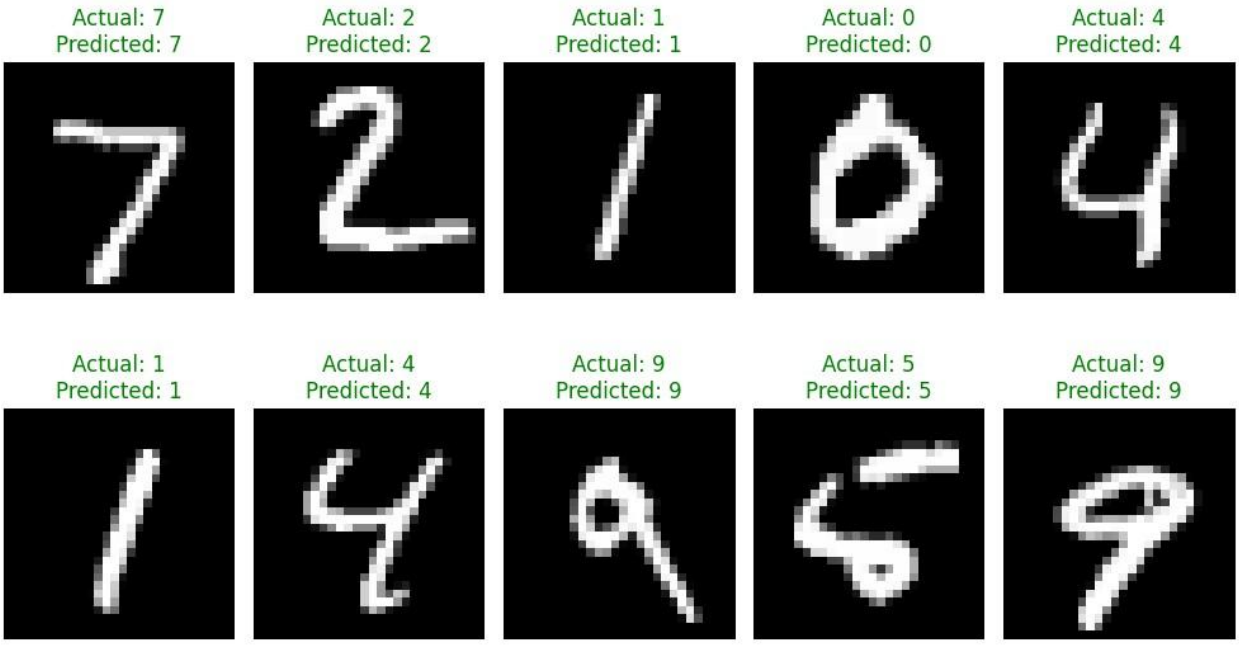
```
    actual = actual_labels[i]
    predicted = predicted_labels[i]

    color = 'green' if actual == predicted else 'red'

    plt.title(
        f"Actual: {actual}\nPredicted: {predicted}",
        color=color
    )
    plt.axis('off')

plt.tight_layout()
plt.show()

# Display prediction comparison
print("Actual Labels:   ", actual_labels)
print("Predicted Labels:", predicted_labels)
print("Correct Predictions:", np.sum(actual_labels == predicted_labels),
      "out of", num_images)
```




```
Actual Labels:    [7 2 1 0 4 1 4 9 5 9]
Predicted Labels: [7 2 1 0 4 1 4 9 5 9]
Correct Predictions: 10 out of 10
```

9. Analyze the model's performance, identify possible misclassifications, and discuss factors affecting classification accuracy.

Performance Analysis

The CNN model was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The model's predictions on unseen test images were compared with their actual labels to identify correct and incorrect classifications.

Misclassifications may occur when handwritten digits have similar shapes, such as 3 and 5, 4 and 9, or 7 and 9. Variations in handwriting style, unclear strokes, and image quality can also affect the predictions.

The main factors affecting classification accuracy are:

1. **Dataset Quality:** Poor-quality or ambiguous images can lead to incorrect predictions.
2. **Training Data:** Insufficient or unrepresentative training samples may reduce generalization.
3. **Model Architecture:** The number of convolutional layers, filters, and neurons affects feature extraction.
4. **Hyperparameters:** Learning rate, batch size, and number of epochs influence model performance.
5. **Overfitting:** A model may perform well on training data but poorly on unseen images.
6. **Image Preprocessing:** Incorrect normalization or inconsistent image dimensions can affect accuracy.

Overall, CNNs can effectively classify handwritten digits by automatically learning image features. Analyzing misclassified images and evaluation metrics helps identify areas for improvement.

10. Summarize the results and discuss the practical applications of CNN-based image classification in modern computer vision systems. Results

In this experiment, a Convolutional Neural Network (CNN) was designed, trained, and evaluated using the MNIST handwritten digit dataset. The model learned important image features through convolutional and pooling layers and classified handwritten digits into ten classes, from 0 to 9.

The model's performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. Predictions on unseen test images helped compare the predicted labels with the actual labels and identify misclassifications. The results demonstrate the effectiveness of CNNs in handwritten digit classification.

Practical Applications

1. **Handwritten Digit Recognition:** Used to recognize digits on forms, examination papers, and handwritten documents.
2. **Optical Character Recognition (OCR):** Helps convert scanned documents and printed text into machine-readable formats.
3. **Face Recognition:** Used in identity verification, access control, and biometric authentication.

4. **Medical Image Classification:** Helps classify medical images to support disease detection and diagnosis.
5. **Object and Image Recognition:** Used in surveillance, robotics, and intelligent vision systems.
6. **Industrial Automation:** Helps inspect products and identify defects in manufacturing processes.

Conclusion

The experiment demonstrates that CNNs can automatically extract useful image features and perform image classification effectively. Their ability to learn spatial patterns makes them an important technique in modern computer vision applications.

Questions and Answers

1. What is image classification? How does it differ from object detection and image segmentation?

Image classification is a computer vision task in which an image is assigned a class label, such as a cat, dog, or digit.
The differences are:

- **Image Classification:** Assigns a label to the entire image.
- **Object Detection:** Identifies objects and locates them using bounding boxes.
- **Image Segmentation:** Classifies individual pixels to identify the exact region occupied by an object.
  Example: For an image containing a dog and a cat, classification identifies the main image category, object detection locates both animals, and segmentation marks the pixels belonging to each animal.

2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

A **Convolutional Neural Network (CNN)** is a deep learning model designed primarily for processing images and learning spatial features automatically.
Its main components are:

1. **Input Layer:** Receives the image as pixel values.
2. **Convolutional Layer:** Extracts features such as edges, textures, and shapes using filters.
3. **Activation Function:** ReLU introduces non-linearity into the network.
4. **Pooling Layer:** Reduces the spatial dimensions of feature maps.
5. **Flatten Layer:** Converts feature maps into a one-dimensional vector.
6. **Fully Connected Layer:** Learns relationships between extracted features.
7. **Output Layer:** Produces class probabilities using Softmax for multiclass classification.

Working: The image passes through convolutional and pooling layers to extract features. These features are then processed by fully connected layers to predict the final class. During training, the model updates its weights using backpropagation and an optimizer to minimize the loss.

3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

- **Convolutional Layers:** Apply filters to images to extract important features such as edges, corners, textures, and patterns. **Pooling Layers:** Reduce
- the spatial size of feature maps, lower computational cost, and help retain important features. Max pooling selects the maximum value from a local region.
- **Fully Connected Layers:** Combine extracted features and use them to make the final classification.

Together, these layers enable a CNN to learn useful image representations and classify images effectively.

4. Why is image normalization performed before training a deep learning model?

Image normalization scales pixel values to a smaller, consistent range, commonly from 0–255 to 0–1 by dividing each pixel value by 255.
Its benefits are:

1. Improves numerical stability during training.
2. Helps the optimizer converge more efficiently.
3. Prevents large pixel values from dominating computations.
4. Provides a consistent input scale for the model.

For example, a pixel value of 128 becomes approximately 0.502 after normalization. In the MNIST experiment, normalization prepares grayscale images for CNN training.

5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

ReLU (Rectified Linear Unit):
ReLU introduces non-linearity into the network, allowing it to learn complex patterns. It returns the input when it is positive and zero otherwise.
Formula: ReLU(x) = max(0, x) **Softmax:**

Softmax converts output scores into probabilities across multiple classes. The probabilities sum to 1, allowing the model to select the most likely class.

Example: In digit classification, Softmax produces probabilities for digits 0–9, and the class with the highest probability is selected as the prediction.

6. What is a Confusion Matrix? How is it used to evaluate classification performance?

A **Confusion Matrix** is a table that compares actual class labels with predicted class labels.
For binary classification, it contains four values:

- **True Positive (TP):** Positive cases correctly predicted as positive.
- **True Negative (TN):** Negative cases correctly predicted as negative.
- **False Positive (FP):** Negative cases incorrectly predicted as positive.
- **False Negative (FN):** Positive cases incorrectly predicted as negative.
- The confusion matrix helps identify the types of classification errors made by a model. It is also used to calculate accuracy, precision, recall, and F1-score.

For multiclass classification, such as MNIST digit recognition, each row and column represents a digit class. The diagonal shows correct predictions, while off-diagonal values indicate misclassifications.

7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

These are standard metrics used to evaluate classification models.

- **Accuracy:** The proportion of all predictions that are correct.
- **Precision:** The proportion of predicted positive cases that are actually positive.
- **Recall:** The proportion of actual positive cases correctly identified.
- **F1-Score:** The harmonic mean of precision and recall.
- Formulas:

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
Example: Suppose a model identifies spam emails and produces TP = 40, TN = 50, FP = 10, and FN = 5.

- Accuracy = 90/105 = 85.71%
- Precision = 40/50 = 80%
- Recall = 40/45 = 88.89%
- F1-Score ≈ 84.21%
-

These metrics provide different perspectives on model performance. Accuracy measures overall correctness, while precision and recall focus on positive predictions and actual positive cases.

8. Compare traditional feature-based image classification methods (e.g., SIFT/HOG + SVM) with CNN-based classification.

Traditional Methods:

- Use handcrafted features such as SIFT or HOG.
- Require separate feature extraction and classification stages.
- Often work well on smaller datasets and simpler tasks.
- Depend heavily on the quality of manually designed features.

CNN-Based Methods:

- Automatically learn image features from training data.
- Combine feature extraction and classification in one trainable model.
- Can learn complex patterns and hierarchical representations.
- Usually require more training data and computational resources.

Conclusion: Traditional methods can be effective for simpler tasks with limited data, whereas CNNs generally perform better on complex image classification problems when sufficient data and computing resources are available.

9. Mention five real-world applications of CNN-based image classification.

Healthcare: Classifying medical images to support disease detection.

Face Recognition: Identifying or verifying individuals in biometric systems.

OCR: Recognizing handwritten digits and characters in documents.

Agriculture: Classifying plant diseases from leaf images.

Industrial Inspection: Identifying defective products in manufacturing.

CNNs are widely used because they can automatically learn relevant visual features from images.

10. How can Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?

1. Data Augmentation: Creates modified training images through transformations such as rotation, shifting, cropping, and flipping where appropriate. It increases training diversity and can reduce overfitting.

2. Transfer Learning: Uses a model pretrained on a large dataset and adapts it to a new classification task. It can reduce training time and improve performance when the target dataset is small.

3. Hyperparameter Tuning: Selects suitable values for parameters such as learning rate, batch size, number of epochs, filters, and network layers. Proper tuning can improve convergence, accuracy, and generalization.

Conclusion: These techniques can improve the accuracy and reliability of CNN-based image classification models when applied appropriately to the dataset and task.